

Lab 3: Component Modelling & Architectural Pattern Selection

Software Engineering Laboratory

Name	B Varun
SRN	PES1UG24CS109
Section	B
Problem Statement (Lab 1)	Disaster Relief Supply & Volunteer Coordinator

1. System Overview

The Disaster Relief Supply & Volunteer Coordinator is an emergency-response platform for coordinating shelter supply needs, volunteer resources and relief deliveries. A Relief Volunteer can submit or update field information, while a Disaster Manager prioritizes supply requisitions, assigns suitable volunteers and monitors the relief operation. The platform also maintains delivery status and disaster information.

The architecture must also support the Lab 1 non-functional requirements: low network bandwidth consumption with offline GIS caching, and protection of volunteer personal data and shelter location data using encryption and role-based access control.

2. Components Identified

Component	Layer	Responsibility
Volunteer / Manager UI	Presentation	Screens for submitting requisitions, managing availability, viewing assignments, delivery status and reports.
Disaster Dashboard	Presentation	Shows disaster status, shelter needs, delivery state and operational summary.
Application Facade	Business	Single entry point for UI requests and routing to the required business services.
Requisition & Priority Service	Business	Validates field requisitions, checks supply availability and assigns delivery priority.
Volunteer Management Service	Business	Maintains volunteer skills and availability and supports suitable assignments.
Dispatch & Delivery Tracking Service	Business	Creates dispatch records and tracks vehicle delivery state.
Access Control & Authentication	Business	Authenticates users and enforces Relief Volunteer vs. Disaster Manager permissions.
GIS & Disaster Status Service	Business	Maintains disaster status and low-bandwidth/offline GIS information.
Reporting & Analytics Service	Business	Produces operational summaries and historical relief/delivery information.
Data Access Component	Data	Single persistence gateway used by business components for domain data.
Relief Management Database	Data	Stores requisitions, volunteers, assignments, deliveries and disaster status.
GIS / Map Repository	Data	Stores map/reference data used by the dashboard and tracking components.
Warehouse / Logistics Adapter	Integration	Connects the application to central warehouse/logistics information.

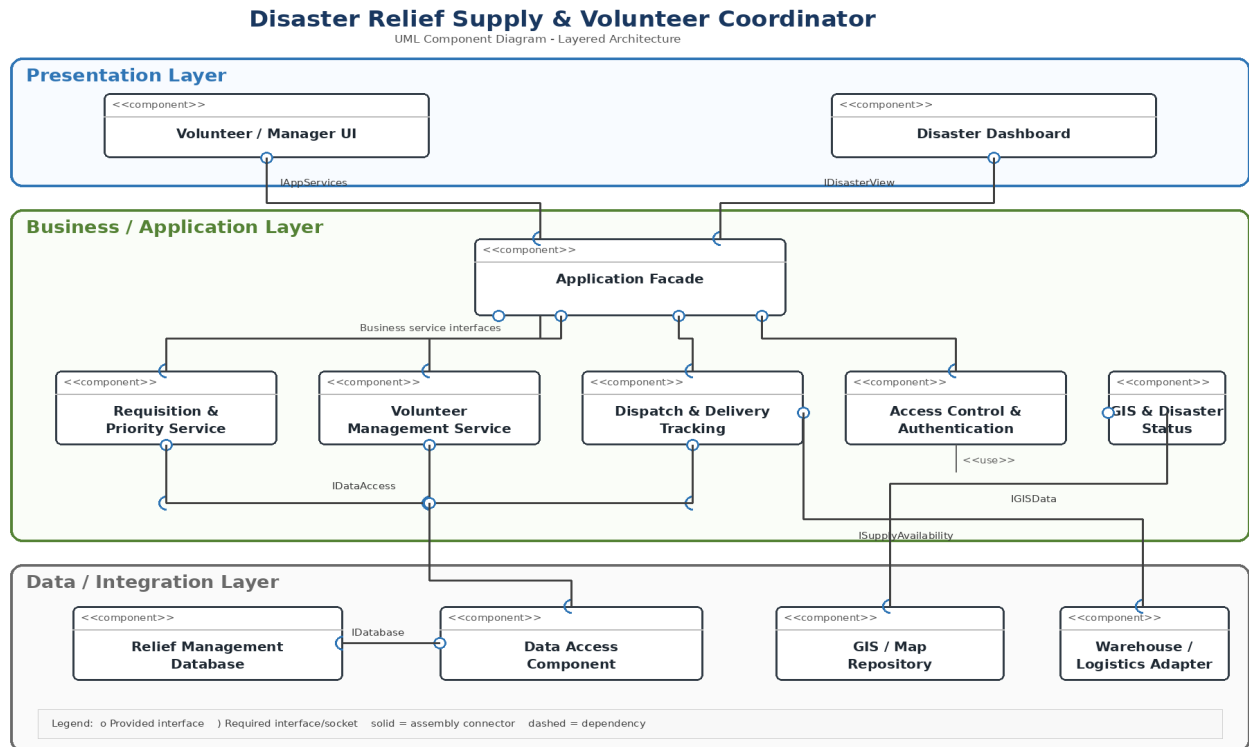
3. Interfaces

The interfaces below identify the principal services provided and required by the components in the UML model.

Interface	Provided by	Required by	Purpose
IAppServices	Application Facade	Volunteer / Manager UI; Disaster Dashboard	Main application requests entering the system.
IRequisition	Requisition & Priority Service	Application Facade	Create, validate and prioritize shelter supply requisitions.
IVolunteer	Volunteer Management Service	Application Facade; Requisition & Priority Service	Find and maintain suitable volunteer skills and availability.
IRole	Access Control & Authentication	Application Facade; UI components	Authenticate users and enforce role permissions.
IDataAccess	Data Access Component	Business services	Read and write domain records through one persistence boundary.
IDeliveryData	Data Access Component	Dispatch & Delivery Tracking Service	Persist and retrieve dispatch and delivery status.
IGISData	GIS / Map Repository	GIS & Disaster Status Service	Supply map/reference data for caching and visualization.
ISupplyAvailability	Warehouse / Logistics Adapter	Requisition & Priority Service; Dispatch & Delivery Tracking	Check available supplies and logistics information.
IReportData	Data Access Component	Reporting & Analytics Service	Read operational records for summaries and reports.
IDatabase	Relief Management Database	Data Access Component	Database connection and persistence operations.

Notation note: a circle represents a provided interface and a socket represents a required interface. Assembly connectors show the interaction between matching interfaces.

4. Component Diagram



The component connections are routed on separate paths so that interfaces, labels and dependencies remain visible and do not overlap.

5. Architectural Justification

Architecture Selection: We chose **Layered Architecture** for the Disaster Relief Supply & Volunteer Coordinator System, organized into Presentation, Business/Application, and Data/Integration layers.

Reason 1 - Clear separation of emergency-response responsibilities

The Lab 1 workflow separates naturally into user interaction, business decisions and persistent information. The presentation layer handles volunteer/manager interaction and disaster visualization; the business layer performs requisition prioritization, volunteer matching, dispatch tracking and access control; and the data layer stores relief, GIS and operational records. This makes the modules easier to understand, test and modify independently.

Reason 2 - Direct support for low-connectivity operation

The Lab 1 requirement explicitly calls for low network bandwidth consumption and offline GIS data caching. Placing GIS caching and disaster-status handling behind the business layer allows the application to reuse locally cached map/reference information while keeping network-dependent integration separate from the user interface.

Security Advantage

Layering provides a controlled access boundary through the Access Control & Authentication component. A Relief Volunteer and a Disaster Manager can be authenticated and authorized before requests reach business services or sensitive data. The presentation layer does not access the database directly, reducing the risk of bypassing role checks and supporting the Lab 1 requirement for protecting volunteer personal data and shelter location data.

Performance Benefit

Offline GIS caching reduces repeated network requests in disaster areas with poor connectivity. Separating data access also allows frequently used operational and map data to be cached or indexed without changing the user-facing components. Business services can additionally reuse cached disaster/reference information to avoid unnecessary database work.

Final Decision

Layered Architecture provides a straightforward decomposition for this problem statement while addressing the documented usability, security and performance constraints. Microservices could provide independent scaling and fault isolation, but would add distributed-system complexity that is not required by the stated scope. Client-Server would centralize access but gives less guidance for decomposing the application responsibilities represented in this component model.